Clinical Trial Dashboard

Name: Christopher Florentino
Date: 5/29/26

Project Overview

Build a data analysis and visualization dashboard for exploring immune cell population data from a clinical trial. The dashboard loads raw cell-count CSV data into a relational database, runs statistical analyses, and presents results in an interactive web application. The primary research question is whether baseline immune cell composition differs between patients who respond to the drug candidate **miraclib** and those who do not.

Project Requirements

Part 1 — Data Ingestion

- Parse a CSV file containing cell-count measurements across subjects, samples, and immune cell populations
- Normalize the flat CSV into a relational schema (subjects, projects, samples, cell_counts)
- Load records into a local SQLite database with proper foreign key constraints
- Support idempotent re-runs (INSERT OR IGNORE for reference tables; full reload for cell_counts)

Part 2 — Cell Population Frequency Overview

- Compute the relative frequency (%) of each immune cell population per sample using a SQL window function
- Expose an interactive stacked bar chart filterable by sample
- Expose an interactive frequency table filterable by sample and/or population

Part 3 — Responders vs Non-Responders Statistical Analysis

- Restrict the cohort to: Melanoma condition · miraclib treatment · PBMC sample type
- For each cell population, run a two-sided Mann-Whitney U test comparing responder vs non-responder frequencies
- Report median frequencies, U statistic, p-value, and significance flag per population

- Visualize distributions as side-by-side box plots with jittered data points and p-value annotations

Part 4 — Baseline Subset Analysis

- Further restrict to samples collected at time_from_treatment_start = 0 (pre-treatment baseline)
- Compute summary metrics: total samples, total subjects, responder/non-responder counts, sex distribution
- Visualize per-project sample counts and a project → response → sex sunburst breakdown
- Display the raw baseline records in a sortable, filterable data table

Tech Stack

Layer	Technology
Language	Python 3.9+
Web Framework	Dash 2.x (Plotly)
Charting	Plotly Express / Plotly Graph Objects
Data Processing	pandas, NumPy
Statistics	SciPy (mannwhitneyu)
Database	SQLite 3
Query Layer	Raw SQL via sqlite3 + pandas.read_sql_query

Data Model

subjects

Column	Type	Notes
subject	TEXT	Primary key
age	INTEGER	
sex	TEXT	M or F
condition	TEXT	e.g. melanoma, carcinoma
response	TEXT	yes, no, or NULL

projects

Column	Type	Notes
project	TEXT	Primary key

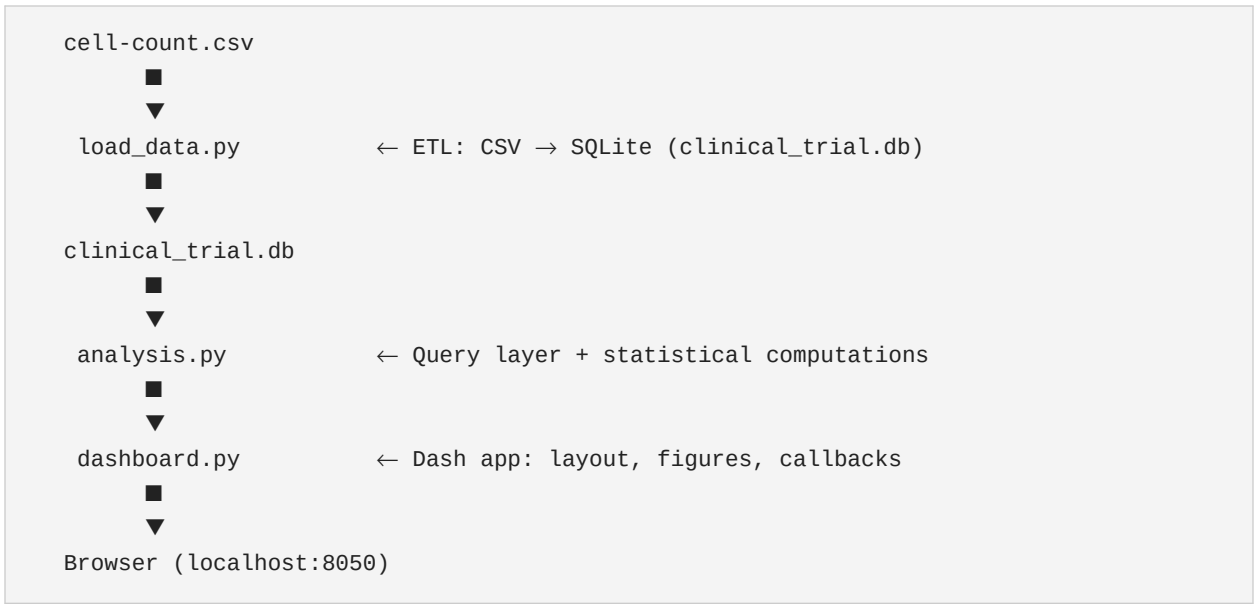
samples

Column	Type	Notes
sample	TEXT	Primary key
subject	TEXT	FK → subjects
project	TEXT	FK → projects
sample_type	TEXT	e.g. PBMC
treatment	TEXT	e.g. miraclib
time_from_treatment_start	INTEGER	Days; 0 = baseline

cell_counts

Column	Type	Notes
id	INTEGER	Primary key (autoincrement)
sample	TEXT	FK → samples
population	TEXT	b_cell, cd8_t_cell, cd4_t_cell, nk_cell, monocyte
count	INTEGER	Raw cell count

Architecture



Module Design

load_data.py

Responsible for all ETL work. Connects to (or creates) **clinical_trial.db**, initializes the schema, then ingests the CSV.

Function	Description
init_db(conn)	Creates the four tables if they do not already exist
load_csv(conn, path)	Reads the CSV, deduplicates reference rows, inserts subjects/projects/samples with INSERT OR

analysis.py

Stateless query and computation layer. Every function opens and closes its own connection so it is safe to call from any context.

Function	Returns	Description
get_frequency_table()	DataFrame	All samples × populations with raw count and relative frequency (%) computed via
get_responder_data()	DataFrame	Frequency data restricted to melanoma · miraclib · PBMC cohort, joined with subj
run_statistics()	DataFrame	Mann-Whitney U test results for each population, sorted by p-value ascending
get_baseline_subset()	dict	Pre-treatment (t=0) samples with aggregated summary tables for subjects, respon
make_boxplot()	plt.Figure	Matplotlib boxplot figure (static export alternative to the interactive Dash version)

Statistical Method: Mann-Whitney U (two-sided)

- Non-parametric; no normality assumption required
- Null hypothesis: the two distributions (responder vs non-responder frequency) are identical
- Significance threshold: $p < 0.05$
- Annotation key: *** $p < 0.001$ · ** $p < 0.01$ · * $p < 0.05$ · ns otherwise

dashboard.py

Dash application with three tabs. All figures are rendered with Plotly for interactivity. The app loads all four analysis results at startup and uses callbacks to filter/re-render on user interaction.

Tabs

Tab ID	Label	Content
tab-overview	Data Overview	Filterable stacked bar chart + frequency DataTable
tab-stats	Statistical Analysis	Box plots, cohort callout, statistics DataTable
tab-subset	Subset Analysis	Metric cards, bar chart, sunburst, raw record DataTable

Callbacks

Output	Inputs	Behavior
freq-bar figure	sample-filter	Redraws stacked bar for selected samples only
freq-table children	sample-filter, pop-filter	Redraws DataTable filtered by sample and/or population
tab-content children	tabs value	Swaps the entire tab body on tab change

Color Palette

Key	Hex	Used For
HEADER_COLOR	#1a2c45	Headers, DataTable header backgrounds
ACCENT	#2980b9	Section underlines, tab highlight
BG	#f0f2f5	Page background
CARD_BG	#ffffff	Card surface
Responder	#27ae60	Green — responder data series
Non-Responder	#e74c3c	Red — non-responder data series

Example Queries

Frequency Table

```
SELECT
    cc.sample,
    cc.population,
    cc.count,
    SUM(cc.count) OVER (PARTITION BY cc.sample) AS total_count,
    ROUND(
        100.0 * cc.count / SUM(cc.count) OVER (PARTITION BY cc.sample),
        4
    ) AS percentage
FROM cell_counts cc
ORDER BY cc.sample, cc.population;
```

Responder Cohort

```
SELECT
    cc.sample,
    su.subject,
    su.response,
    cc.population,
    cc.count,
    ROUND(
        100.0 * cc.count / SUM(cc.count) OVER (PARTITION BY cc.sample),
        4
    ) AS percentage
FROM cell_counts cc
JOIN samples sa  ON cc.sample  = sa.sample
JOIN subjects su ON sa.subject = su.subject
WHERE su.condition   = 'melanoma'
  AND sa.treatment   = 'miraclib'
  AND sa.sample_type = 'PBMC'
  AND su.response   IS NOT NULL
ORDER BY cc.sample, cc.population;
```

Example Statistical Output

```
[
  {
    "population":             "cd8_t_cell",
    "n_responders":           18,
    "n_non_responders":       22,
    "median_responders":      24.31,
    "median_non_responders": 18.74,
    "mann_whitney_u":         284,
    "p_value":                0.01200,
    "significant (p<0.05)":   true
  },
  {
    "population":             "b_cell",
    "n_responders":           18,
    "n_non_responders":       22,
    "median_responders":      11.20,
    "median_non_responders": 12.05,
    "mann_whitney_u":         201,
    "p_value":                0.48300,
    "significant (p<0.05)":   false
  }
]
```


Running the Project

```
# 1. Install dependencies
pip install -r teiko_proj/requirements.txt

# 2. Load data into SQLite
python teiko_proj/load_data.py

# 3. Launch the dashboard
python teiko_proj/dashboard.py
# → Open http://localhost:8050
```